

Implementácia Entity Component System architektúry

Branislav Trstenský

Tomáš Miština

1. decembra 2025

Obsah

Abstrakt	1
1 Úvod	2
1.1 Entity	2
1.2 Komponenty	2
1.3 Systémy	2
1.4 Komunikácia medzi systémami	3
2 Porovnanie s Objektovo Orientovaným Programovaním (OOP)	3
2.1 Ďalšie Výhody ECS	4
3 Implementačné prístupy	4
3.1 Archetypový systém	4
3.2 Pred-filtrovaný zoznam	5
4 Popis projektu	6
4.1 Implementácia	6
4.2 Reprezentácia objektov	7
5 Záver	9
Literatúra	9

Abstrakt

Entity-Component-System(ECS) je architektúra používaná pre vývoj hier. Tento vzor uprednostňuje kompozíciu, t.j. získavanie funkcionality pridávaním komponentov, na rozdiel od objektovo orientovaných systémov, kde sa funkcionality pridáva dedičnosťou. Takto ECS prispieva k modularite, predchádza redundanciami a zjednodušuje rozšírenie kódu.

V ECS sú herné objekty reprezentované prostredníctvom entít, ktoré samy o sebe nemajú žiadne dáta ani funkcionality. Ku každej entite je možné pripojiť množinu komponentov, ktoré iba udržiavajú stav. Funkcionality je implementovaná prostredníctvom systémov. Každý systém definuje množinu komponentov, ktoré vyžaduje a následne je tento systém použitý na všetky entity, ktoré dané komponenty obsahujú.

Bude detailne opísaný ECS a jeho časti. Budú vysvetlené rôzne spôsoby jeho implementácie, s porovnaním ich výhod a nevýhod. Predstavia sa rôzne možnosti modelovania herných objektov podľa ECS a budú ukázané rozdiely oproti objektovo orientovanému systému.

Bude implementovaný herný engine, postavený na architektúre ECS a na demonštráciu jeho funkcionality bude implementovaná jednoduchá hra. Herný engine bude určený pre spustenie v prehliadači prostredníctvom jazyku JavaScript. Naša implementácia bude porovnaná s existujúcimi riešeniami. V rámci implementácie bude vytvorená funkcionality pre manažment entít a ich spojenia s komponentami. Bude vytvorená funkcionality pre definíciu systémov a bude implementovaná efektívna metóda vyhľadávania

entít, ktoré majú množinu komponentov aplikovateľnú pre daný systém. Taktiež budú vytvorené systémy pre ovládanie hráča, detekciu kolízií, vykresľovanie objektov na obrazovku a autonómne správanie sa nepriateľov v hre.

1 Úvod

Entity Component System (ECS) je softvérový architektonický vzor, ktorý sa najčastejšie používa vo vývoji počítačových hier na reprezentáciu objektov herného sveta. Tento vzor uprednostňuje kompozíciu, t.j. získavanie funkcionality pridávaním komponentov. Ako už jeho názov naznačuje, skladá sa z troch základných prvkov, teda z entít, komponentov a systémov. Entity slúžia ako reprezentácia herných objektov, komponenty definujú vlastnosti týchto entít a uchovávajú ich stav. Systémy následne vykonávajú herné procesy a implementujú správanie sa entít.

1.1 Entity

Entity sú jedinečné identifikátory bez vlastných dát, alebo správania. Každá entita predstavuje odlišný objekt v systéme (napr. hráča, nepriateľa alebo predmet), ale sama o sebe neobsahuje žiadne informácie. Slúžia ako spôsob zoskupovania alebo spájania komponentov. Poskytujú spôsob, ako odkazovať na objekty neutrálnym spôsobom, čo umožňuje dynamické skladanie.

Implementačne môžu byť entity realizované rôznymi spôsobmi, napríklad ako:

- **Integer:** interný stav entity je spravovaný mimo používateľského kódu, napríklad v poli alebo hašovacej mape.
- **Opaque pointer:** pointer na interný objekt, ku ktorému nemá používateľ priamy prístup.

Entita sa zo strany používateľa používa výlučne ako parameter pre funkcie ECS na manipuláciu s príslušnými entitami.

1.2 Komponenty

Komponent predstavuje určitú vlastnosť, ktorú entita môže získať, a je zodpovedný za udržiavanie stavu relevantného k tejto vlastnosti. Každý komponent predstavuje práve jeden aspekt alebo atribút, ako napríklad polohu, rýchlosť, stav a podobne. Neobsahujú žiadne správanie ani logiku, sú iba pasívne nosiče údajov. Umožňujú kompozíciu namiesto dedičnosti, čo umožňuje flexibilné a prispôsobiteľné definície entít bez rigidných hierarchií tried.

Existujú aj prázdne komponenty, bez stavu, ktoré sa používajú na označenie entít, napríklad že entita je nepriateľ a mal by dostať poškodenie od hráča, atď. Tieto komponenty sú tzv. tagy. Kombináciou rôznych komponentov sa definujú komplexné herné objekty. Napríklad, entita reprezentujúca nepriateľa by bola charakterizovaná komponentmi ako *Pozícia*, *Život*, *Hitbox* a *Sprite*.

1.3 Systémy

Systémy zapuzdrujú logiku a správanie v ECS a pôsobia na entity spracovaním ich pridružených komponentov. Čítajú a zapisujú do údajov komponentov a vykonávajú rôzne úlohy, ako napríklad aktualizácie pohybu, detekcia kolízií či vykresľovanie. Sú globálne definované a oddelené od údajov, čo podporuje modularitu a jasné oddelenie úloh.

Systémy môžu mať priradenú prioritu, čo je kľúčové pre zabezpečenie správneho poradia vykonávania (napr. fyzika musí byť aktualizovaná pred vykresľovaním). Toto je dôležitý kontrast oproti objektovo orientovaným systémom, kde môže byť vynútenie takýchto závislostí implementačne náročnejšie.

Každý systém definuje tzv. query (dotaz), ktorý určuje, akú minimálnu množinu komponentov musí entita vlastniť, aby na ňu mohol byť systém aplikovaný. Niektoré enginy podporujú v query aj možnosť určiť, ktoré komponenty nemôžu byť v entite, čo síce komplikuje vyhľadávanie avšak prináša väčšiu flexibilitu.

Tieto query sa môžu použiť napríklad v systéme, ktorý musí spracovať entitu len raz. Systém pridá do entity nejaký komponent, ktorý mu naznačí, že už znovu na ňu nemá byť aplikovaný.

Niektoré enginy rozlišujú aplikáciu systémov na readonly alebo mutable, kde readonly systémy nemôžu meniť stav komponentov. Táto vlastnosť sa používa pri paralelizácii, kde na jednej entite môže byť naraz spustených viac readonly systémov.

1.4 Komunikácia medzi systémami

Pre zaručenie správnej funkcionality každého ECS enginu je potrebné aby jednotlivé systémy vedeli navzájom medzi sebou komunikovať. Samotné systémy však zvyčajne medzi sebou nekomunikujú priamo. Namiesto toho sa komunikácia alebo koordinácia medzi systémami deje nepriamo prostredníctvom zdieľaných údajov komponentov alebo cez udalosti. Existuje viacero spôsobov nepriamej komunikácie, ktorú môžu systémy využiť:

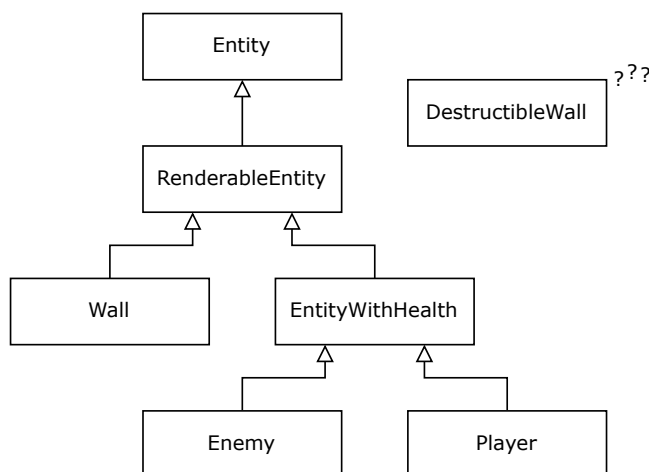
- **Nepriama komunikácia prostredníctvom komponentov**
 - Systémy čítajú a upravujú zdieľané komponenty pripojené k entitám. Jeden systém aktualizuje údaje komponentov a druhý systém číta tieto aktualizované údaje. Zmeny v stave komponentov slúžia ako signály, na ktoré môžu ostatné systémy reagovať, čo efektívne umožňuje koordináciu.
- **Nepriama komunikácia pomocou eventov**
 - Systémy môžu vysielat' udalosti (napr. „hráč utrpel poškodenie“), na ktoré sa ostatné systémy prihlásia a reagujú na ne. To umožňuje asynchrónnu alebo oddelenú komunikáciu bez priamych systémových závislostí. Existujú dva hlavné spôsoby komunikácie prostredníctvom eventov, na základe spôsobu ich implementácie:
 - **Event bus**, v ktorom sa systémy na začiatku prihlásia na všetky eventy, ktoré chcú dostávať. Keď systém vytvára event, spustí sa funkcia na všetkých prihlásených systémoch. Je to však úplne separátne od ECS.
 - **Event komponent**, čo je event viazaný na entitu. Keď systém vytvára event pridá sa komponent, napr. Damage-Event-Component. Komponent je dočasný a po skončení processing je odstránený. Toto konštantné pridávanie a odoberanie komponentov môže byť pre niektoré implementácie ECS náročné ale používa existujúce koncepty ECS.
 - **Hybridný systém**: je použitý event komponent, ale pre zmenšenie dopadu častého pridávania alebo odoberania komponentov, je tento komponent explicitne označený a udržiavaný v separátnej dátovej štruktúre schopnej rýchlej modifikácie.
- **Zdieľané zdroje alebo služby**
 - V komplexných architektúrach ECS môžu zdieľané objekty služieb alebo správcovia (napr. audio systém, vstupný systém) poskytovať nepriame komunikačné prostriedky. Tieto zdieľané zdroje umožňujú systémom požadovať alebo upozorňovať na udalosti. Napríklad služba používateľského vstupu poskytuje informácie o stave klávesnice alebo myši.

2 Porovnanie s Objektovo Orientovaným Programovaním (OOP)

Hlavnou výhodou ECS je skladateľnosť (composability). V tradičnom OOP by sa herné objekty často reprezentovali hierarchiou tried, kde triedy dedia funkcionality od nadtried a pridávajú špecifickú funkcionality. Pri pokuse o definovanie nového, komplexného objektu, ktorý má množinu vlastností nekompatibilných z existujúcou hierarchiou, je potrebné použiť jedno z nasledujúcich neoptimálnych riešení:

- Potreba rozsiahlej reorganizácie triednej hierarchie.
- Duplikácia implementácie identickej funkcionality v rôznych vetvách hierarchie.
- Vloženie všeobecnej vlastnosti do príliš vysokej nadtriedy, čím sa pridá funkcionality do tried, ktoré ju nepotrebujú.

V ECS, keďže každá vlastnosť je samostatný komponent, je komplexný objekt je definovaný jednoduchým výberom a kombináciou potrebných komponentov.



Obr. 1: Príklad problematickej hierarchie tried. V tomto prípade nie je možné definovať objekt `DestructibleWall` bez rozsiahlych zmien.

2.1 Ďalšie Výhody ECS

- **Rozšíriteľnosť:** Keďže systémy definujú len požadované vlastnosti, je možné pridávať nové funkcionality bez nutnosti modifikácie existujúcich entít. Napríklad, pridaním systému pre zobrazenie ukazovateľa zdravia nie je potrebná žiadna zmena v entitách alebo komponentoch, iba pridanie nového systému.
- **Minimalizácia dynamic dispatch:** v OOP by každý objekt musel predefinovať metódu update, čo by si vyžadovalo volanie virtuálnej metódy pre každý objekt. V ECS sa realizuje iba jedno volanie pre celý systém, čo prispieva k vyššiemu výkonu.
- **Jednoduchšia paralelizácia:** je možné vopred určiť, ktoré systémy nepracujú s rovnakými dátami a spúšťať ich paralelne. Dokonca, ak systémy nemodifikujú stav komponentov, môžu byť bezpečne vykonávané súčasne aj nad rovnakými entitami.
- **Definovanie poradia spracovania:** keďže spracovanie prebieha na úrovni systémov, je oveľa jednoduchšie definovať ich poradie podľa priority. Poľe systémov je možné jednoducho zoradiť pri počiatkovej registrácii. V OOP by bolo potrebné zoradovať pole všetkých herných objektov pri každom pridaní alebo odobratí objektu

3 Implementačné prístupy

Už v opise systémov bolo spomenuté, ako veľmi dôležité sú query pre správne fungovanie každého systému. Preto záleží na tom, akým spôsobom sú query implementované. Existujú 2 najznámejšie spôsoby implementácie query: implementácia pomocou archetype systému a pomocou pred-filtrovaného zoznamu.

3.1 Archetypový systém

Archetyp predstavuje konkrétnu, unikátnu množinu komponentov. Pre každý archetyp je vyhradené pole, v ktorom sú uložené komponenty entít patriacich k danému archetypu. Každá entita má len jeden archetyp a existuje presne v jednom poli.

Archetypový systém pracuje nasledovne:

- Pri vytvorení entity sa nájde (alebo vytvorí) príslušný archetyp.
- Pri pridaní/odobratí komponentu musí byť entita presunutá do poľa nového archetypu, pričom existujúce komponenty sú skopírované.

Archetyp: {Position, Sprite}

ID	Position	Sprite	ID	Position	Sprite	...
----	----------	--------	----	----------	--------	-----

Archetyp: {Health, Position, Sprite}

ID	Health	Position	Sprite	ID	Health	Position	Sprite	...
----	--------	----------	--------	----	--------	----------	--------	-----

Obr. 2: Vizualizácia archetypového systému. Komponenty archetypov sú uložené v pamäti priamo za sebou.

Pri aplikovaní systému sa nájdu všetky archetypy, ktoré sú nadmnožinou komponentov požadovaných v query systému. Týmto spôsobom sú nájdené všetky aplikovateľné entity, keďže aplikovateľný archetyp je aj nadmnožina potrebných komponentov. V prípade, že sa zničia všetky entity v archetypu, je možné daný archetyp zahodiť.

Tento prístup je určený pre použitie s low-level jazykom a predpokladá že všetky komponenty budú v pamäti uložené jeden za druhým. V JavaScripte, kde všetky objekty sú cez reference a pamäť nemôžeme ovládať manuálne, to predchádza potrebe kopírovať dáta, iba kopírujeme reference, ale odstraňuje to výhodu linearity prístupu k pamäti.

Výhody archetypového systému:

- **Linearita prístupu k pamäti:** komponenty sú v pamäti uložené blízko seba, čo umožňuje lepšie využitie cache pamäte.
- **Rýchla iterácia:** iterácia prebieha priamo cez polia.
- **Rýchle vyhľadávanie:** pre každý systém musíme prehľadávať iba platné archetypy nie cez všetky entity.

Nevýhody archetypového systému:

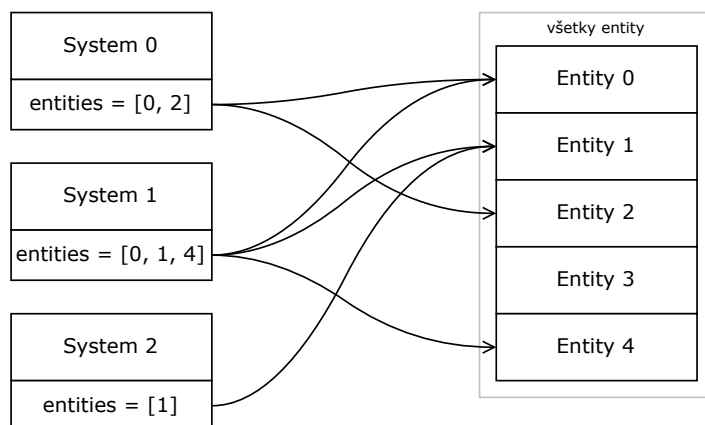
- **Obtiažnosť modifikácie komponentov:** pridanie alebo odobratie komponentu je nákladné z dôvodu nutnosti presunu a kopírovania dát.
- **Modifikácia poľa pri zničení entity:** vyžaduje presun elementov na vyplnenie prázdneho miesta alebo podporu sparse polí.

3.2 Pred-filtrovaný zoznam

V pred-filtrovanom zozname sa pre každý systém udržiava tzv. dopyt, dátová štruktúra s polom všetkých existujúcich entít, ktoré majú komponenty požadované daným systémom.

- Pri vytvorení, pridaní alebo odobratí komponentov entity sa vykoná kontrola voči všetkým existujúcim dopytom systémov a entita sa prípadne pridá alebo odoberie do ich interných polí.
- Pri aplikovaní systému sa jednoducho iteruje cez toto už pripravené pole referencií.

Pretože entita môže existovať vo viacerých zoznamoch (dopytoch), sú v nich uložené iba referencie, zatiaľ čo samotné dáta sú v separátnej dátovej štruktúre. V prípade, že má viac systémov identické požiadavky, je možné znovu použiť jeden dopyt pre všetky, je však potrebné implementovať cache systém.



Obr. 3: Vizualizácia prístupu cez pred-filtrovaný zoznam. Každý systém má dopyt z referenciami na platné entity.

Výhody pred-filtrovaného zoznamu:

- **Jednoduchá modifikácia komponentov:** pretože polia v dopytoch obsahujú iba referencie, ich modifikácia je jednoduchšia.
- **Rýchla iterácia:** pri aplikovaní systému prebieha iterácia cez už pripravené polia referencií.
- **Nie je potrebné vyhľadávanie:** Pole pre každý systém je už pripravené.

Nevýhody pred-filtrovaného zoznamu:

- **Chýba linearita prístupu k pamäti:** pri každom prvku polia dochádza k dereferencovaniu, čo zťažuje využitie cache.
- **Chýba flexibilita zo strany systémov:** potreba preskenovať všetky entity pri definícii nového systému alebo mať všetky systémy definované od začiatku

4 Popis projektu

Výsledkom nášho projektu je implementácia jednoduchkej hry s použitím ECS. Žáner hry je wave survival, kde hráč musí prežiť čo najviac vln nepriateľov, kde každá vlna má väčší počet nepriateľov ako predošlá. Hra je založená na viacnásobnom opakovaní s cieľom dosiahnuť čo najvyššie skóre.

Hra je 2D z horného pohľadu. Dej prebieha v aréne, ktorá limituje pohyb hráča. Taktiež sú v aréne umiestnené zničiteľné steny, ktoré ďalej sťažujú pohyb ale zároveň poskytujú krytie pred nepriateľmi. V každej vlne je vybrané jedno z možných rozložení arény, pre udržanie pozornosti hráča.

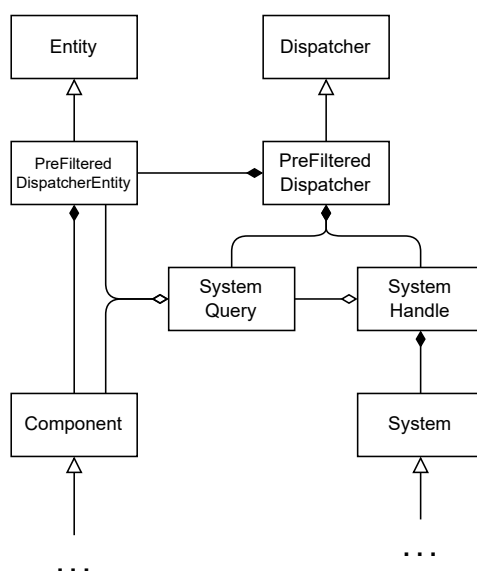
Jedna vlna pokračuje až kým nie sú eliminovaní všetci nepriatelia. Pri prechode do novej vlny sú odstránené steny, vytvorené nové steny podľa vybraného rozloženia a nakoniec sú vytvorení nepriatelia.

Boj medzi nepriateľmi a hráčmi prebieha prostredníctvom projektilov, ktoré odoberajú zdravie zasiahnutých objektov. Používateľ ovláda hernú postavu prostredníctvom klávesnice pre pohyb a myši pre mierenie a strieľanie projektilov.

4.1 Implementácia

Projekt využíva pred-filtrovaný zoznam spolu s hybridným prístupom pre komunikáciu udalostí medzi systémami. Všetky súčasti ECS sú reprezentované abstraktnými triedami, t.j. `Entity`, `Component` a `System`. Taktiež je implementácia ECS reprezentovaná abstraktnou triedou `Dispatcher`. Konkrétnu implementáciu následne tvorí trieda `PreFilteredDispatcher` a jej entita `PreFilteredDispatcherEntity`. Takto je zaručené, že komponenty ani systémy nebudú viazané na vybranú implementáciu ECS.

Komponenty entít sú udržiavané v internom poli triedy `PreFilteredDispatcherEntity`. Pri pridaní systému, je pre jeho dopyt vytvorená trieda `SystemQuery`, ktorá udržiava referencie na tieto komponenty a entity, ktoré ich vlastní. Ak je viacero systémov s rovnakými požadovanými komponentami sú triedy `SystemQuery` zdieľané medzi nimi. Priradenie dopytu a systému je vykonané cez triedu `SystemHandle`, ktorá pri aplikovaní systému volá jeho `update` metódu z vhodnými komponentami získaných z priradenej `SystemQuery`.



Obr. 4: Diagram tried implementácie ECS

Implementácia nepodporuje registráciu systémov už s existujúcimi entitami, pretože toto nie je pre náš prípad použitia potrebné. Vynechanie tejto funkcionality zjednodušuje implementáciu; dokonca nie je vôbec potrebné udržiavať zoznam všetkých entít.

Pre komunikáciu medzi systémami cez udalosti sa používa hybridný prístup, kde sú niektoré systémy predom určené na obsluhu udalostí tak že sú podtriedami `EventHandler`. Udalosti sú komponenty, ktoré sú namiesto pridania k entite cez štandardné API len k nej dočasne priradené pridaním do triedy `EventQuery`, čo je špeciálnym variantom `SystemQuery`, ktorého pole referencií je vymazané pri ukončení spracovania jednej frame. Normálne `SystemQuery` triedy nie sú aktualizované pri tomto procese.

4.2 Reprezentácia objektov

Pre účely hry je definovaných 5 herných objektov:

- Hráč
- Nepriateľ
- Projektil
- Zničiteľná stena
- Stena arény

Na ich realizáciu sú využité nasledovné komponenty:

- `AIState`: udržiava stav umelej inteligencie nepriateľa
- `AttackDamage`: udáva hodnotu koľko života odoberie jeden útok

- **Collider**: označuje objekt ako schopný kolízie a určuje ako jeho kolízia alebo kolízia s ním správa
- **Force**: udáva či je objekt na strane hráča, nepriateľov alebo je neutrálny
- **Health**: hodnota života objektu, taktiež označuje objekt za zraniteľný
- **IsBullet**: označuje objekt ako projektil
- **IsPlayer**: označuje objekt ako hráč
- **MaxSpeed**: udáva rýchlosť pohybu objektu
- **Position**: udáva pozíciu objektu
- **RenderColor**: udáva farbu pre vykresľovanie objektu
- **ShootingCooldown**: udáva čas medzi strelbou projektilu
- **Size**: udáva veľkosť objektu
- **Velocity**: udáva rýchlosť objektu

Herné objekty sú teda zložené nasledovným spôsobom:

- **Hráč**: {Position, Size, RenderColor, IsPlayer, MaxSpeed, ShootingCooldown, AttackDamage, Collider, Health, Force}
- **Nepriateľ**: {Position, Size, RenderColor, MaxSpeed, Health, Collider, Force, AIState, ShootingCooldown, AttackDamage}
- **Projektil**: {Position, Size, RenderColor, Collider, IsBullet, Velocity}
- **Zničitelná stena**: {Position, Size, RenderColor, Health, Force, Collider}
- **Stena arény**: {Position, Size, RenderColor, Collider}

Je možné si všimnúť že jediný rozdiel medzi hráčom a nepriateľom je existencia buď **IsPlayer** alebo **AIState** komponentu. Taktiež je zničitelná stena odvodená od steny arény jednoduchými pridaním komponentov **Health** a **Force**.

Vstup od používateľa je aplikovaný do hry systémami **PlayerMovementSystem** a **PlayerShootingSystem**. Tieto presúvajú objekt hráča a vytvárajú projektily.

Správanie sa nepriateľov je manžované systémami **EnemyBehaviourSystem** a **EnemyShootingSystem**. Tieto systémy reagujú na udalosť **PlayerMovementEvent**, takže nie sú viazané na objekt hráča. Nepriatelia neustále strieľajú v smer hráča a náhodne striedajú medzi typmi pohybu, t.j. medzi krúžením okolo hráča, prenasledovaním a vyhýbaním sa náhodnými pohybmi.

Kolízie medzi objektami sú riešené systémom **CollisionSystem**. Podľa nastavenia komponentu **Collider**, sú kolízie riešené nasledovným spôsobom:

- **static**: bráni prechodu **dynamic** objektov posunutím ich mimo jeho ohraničenia
- **dynamic**: je posunutý mimo ohraničenia **static** objektov
- **trigger**: kolízie nie sú riešené, iba je vytvorená udalosť

Pri každej kolízii je vytvorená udalosť **CollisionEvent**. Táto udalosť sa využíva systémami **EnemyCollisionReactionSystem**, ktorý zmení smer pohybu nepriateľa pri náraze do steny a **BulletHitSystem**, ktorá zničí projektil a zníži život kolidovanému objektu ak je na inej strane konfliktu.

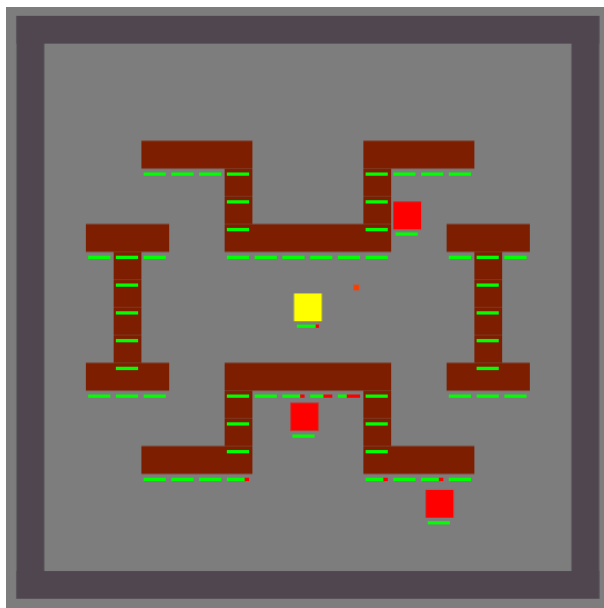
Vykresľovanie objektov a ukazovateľov zdravia je vykonané cez systémy **RenderSystem** a **HealthBarRenderSystem**. Ešte pred vykresľovaním je pohľad vycentrovaný na hráča cez systém **CameraSystem**.

Reakciu na zničenie nepriateľov a nasledovné vytvorenie ďalšej vlny je vykonané systémom **WaveSpawnerSystem**. Tento systém reaguje na udalosť **EntityDiedEvent**, ktorá je vytvorená systémom **DeathSystem**, ktorý ničí objekty, ktorých život sa znížil na nulu.

Systémy sú explicitne zoradené podľa priority. Sumarizovaný zoznam vykonávania systémov v jednej frame je nasledovný:

1. pohyb a strieľanie projektilov hráča

2. posun projektilov
3. vykonávanie správania sa nepriateľov
4. riešenie kolízií objektov
5. reakcia na kolíziu projektilov z objektmi a nepriateľov so stenou
6. vymazanie objektov z nulovým životom
7. vycentrovanie pohľadu na hráča
8. vykresľovanie objektov
9. vykresľovanie ukazovateľov zdravia
10. reakcia na zničenie nepriateľov a následný posun do novej vlny



Obr. 5: Snímka obrazovky hry

5 Záver

ECS je efektívny architektonický vzor, vhodný pre použitie vo je vývoji hier. Skladá sa z troch častí: Entity (identifikátory), Komponenty (dáta) a Systémy (logika). Ponúka významné výhody oproti tradičnému OOP, najmä v oblasti skladateľnosti, rozšíriteľnosti a zjednodušenej paralelizácie.

Literatúra

- [1] BILAS, S., 2002. A Data-Driven Game Object System. 2002. Dostupné z: <https://www.gamedevs.org/uploads/data-driven-game-object-system.pdf>
- [2] MARTIN, A., 3 september 2017. Entity Systems are the future of MMOG development. 3 september 2017. Dostupné z: <https://web.archive.org/web/20250531113805/https://t-machine.org/index.php/2007/09/03/entity-systems-are-the-future-of-mmog-development-part-1/>
- [3] REDMOND, P. et al., október 2025. Exploring the Theory and Practice of Concurrency in the Entity-Component-System Pattern. *Proc. ACM Program. Lang.* október 2025. zv. 9, č. OOPSLA2. DOI 10.1145/3763050. Dostupné z: <https://doi.org/10.1145/3763050>
- [4] SHACKLETT, B. et al., júl 2023. An Extensible, Data-Oriented Architecture for High-Performance, Many-World Simulation. *ACM Trans. Graph.* júl 2023. zv. 42, č. 4. DOI 10.1145/3592427. Dostupné z: <https://doi.org/10.1145/3592427>